

# Image Processing

*HW Author: Mohammad Pashaei*  
*Instructor: Dr. Hadi Amiri*



## Homework

---

### Q1-Exercise: RGB to HSI Conversion Objective:

Understand RGB color channels and convert an RGB image into the HSI color space using Python.

Tasks:

1. Read a color image using PIL.
  2. Separate the R, G, and B channels.
  3. Display each RGB channel in its corresponding color.
  4. Convert the image from RGB to HSI.
  5. Display the H, S, and I channels.
  6. Save all channels as separate image files.
- 

### Q2- In this exercise, you are given one grayscale image that was originally a color image. Your task is to reconstruct a color version of this image.

In addition to the grayscale image, you are also provided with three reference color images that belong to a similar scene or category. These images can be used as references to estimate the color information needed to reconstruct the target image.

Your objectives are:

1. Convert or process the grayscale image to produce a colorized version of the image.
2. Use the information available in the three reference color images to help infer the missing color information.
3. Save the reconstructed color image.

Inputs

- One grayscale target image
- Three reference color images

Output

- One reconstructed color image

---

**Q3- Write a Python function to calculate the PSNR (Peak Signal-to-Noise Ratio) between two color images from scratch, without using any built-in functions or external libraries such as image processing packages. (as before, NumPy and simple lib are allowed)**

### 1. What is PSNR?

PSNR (Peak Signal-to-Noise Ratio) is a common way to measure the difference between two images:

- One image is the **reference** (original, ground truth).
- The other is the **distorted** or **reconstructed** image (e.g., after compression, transmission, denoising, etc.).

PSNR is based on the **Mean Squared Error (MSE)** between the two images.

For 8-bit images (values in  $[0, 255]$ ):

- Let  $MAX_I = 255$  (maximum possible pixel value).
- Compute MSE between the two images.
- Then:

$$PSNR = 10 \log_{10} \left( \frac{MAX_I^2}{MSE} \right) (\text{in dB})$$

Higher PSNR  $\rightarrow$  smaller error  $\rightarrow$  better quality.

### 2. MSE for color images (RGB)

Assume both images are RGB with shape:

$$H \times W \times 3$$

where:

- $H$  = height,
- $W$  = width,
- 3 channels = (R, G, B).

### 2.1. Per-pixel, per-channel difference

For each pixel  $(i, j)$  and channel  $c \in \{R, G, B\}$ :

$$\text{error}(i, j, c) = I_{\text{original}}(i, j, c) - I_{\text{distorted}}(i, j, c)$$

Then square it:

$$\text{error}^2(i, j, c) = (I_{\text{original}}(i, j, c) - I_{\text{distorted}}(i, j, c))^2$$

### 2.2. MSE over all pixels and channels

Total number of scalar elements:

$$N = H \times W \times 3$$

Then:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^H \sum_{j=1}^W \sum_{c=1}^3 \text{error}^2(i, j, c)$$

### 2.3. PSNR from MSE

For 8-bit color images:

$$\text{PSNR} = 10 \log_{10} \left( \frac{255^2}{\text{MSE}} \right)$$

If images are normalized to  $[0, 1]$ , then  $MAX_I = 1$ .

$$\text{PSNR} = 10\log_{10}\left(\frac{1^2}{\text{MSE}}\right) = -10\log_{10}(\text{MSE})$$

## 2.4. Channel-wise vs combined PSNR

There are two common ways:

### 1. One global MSE over all channels (most common):

- Compute one MSE over all  $H \times W \times 3$  values.
- Then compute one PSNR value.

### 2. Per-channel PSNR:

- Compute MSE for R, G, B separately.
- Compute PSNR for each channel.
- Optionally average them.

One of these methods are enough for this home work.

## 3. Important details & pitfalls

### 1. Same size: Both images must have the same dimensions and number of channels.

### 2. Data type:

- Convert to floating point to avoid integer overflow in squaring.

### 3. MSE = 0:

- If images are identical, MSE = 0.
- PSNR is theoretically infinite.
- In code, you must treat this as a special case (e.g., return float('inf')).

### 4. Color space:

- PSNR is often computed in RGB.
- Sometimes in Y channel (luma) of YCbCr, because it correlates more with human perception.

Inputs are two given images.

Output is comparison between PSNR value of your code between 2 photos and PSNR by cv2.PSNR(img1, img2).

---

**Q4-Define type of noise, reduce that, reconstruct the image and calculate PSNR between 2 photos from function you wrote yourself in Q3.**

Use photo given in folder Q4.

---

**Q5-JPEG Compression( Manual calculation )**

In the JPEG compression process, an image is divided into blocks and the Discrete Cosine Transform (DCT) is applied to each block. Assume that the DCT step has already been performed. The resulting DCT coefficients of a block are given below. Your task is to perform the remaining steps used in JPEG compression.

For simplicity, we use a  $4 \times 4$  block instead of the standard  $8 \times 8$ .

Given DCT Coefficient Matrix

$$D = \begin{bmatrix} 145 & -23 & 12 & 2 \\ -18 & 11 & -4 & 0 \\ 8 & -2 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

**1. Quantization**

Use the following quantization matrix  $Q$ :

$$Q = \begin{bmatrix} 10 & 12 & 14 & 18 \\ 12 & 14 & 18 & 24 \\ 14 & 18 & 24 & 32 \\ 18 & 24 & 32 & 40 \end{bmatrix}$$

Compute the quantized matrix  $D_q$  using:

$$D_q = \text{round}\left(\frac{D}{Q}\right)$$

Task:

Compute and write the quantized coefficient matrix  $D_q$ .

## 2. Zig-Zag Scan

Convert the matrix  $D_q$  into a one-dimensional sequence using the standard zig-zag scanning order used in JPEG.

Task:

Write the resulting sequence.

## 3. Run-Length Coding (RLC)

Apply Run-Length Coding to the sequence obtained from the zig-zag scan.

Use the format:

(number of consecutive zeros next non-zero value)

Task:

Write the resulting RLC pairs.

## 4. Conceptual Questions

1. What percentage of the entries in  $D_q$  are zero? What does this indicate about detail removal in JPEG compression?
2. Suppose all elements of the quantization matrix  $Q$  are doubled.

Explain how this change would affect:

- the compression ratio (file size)

- the image quality.

---

END

---

**Please submit your work as follows:**

1. **One Jupyter Notebook** – fully runnable on my device (make sure all file paths and addresses in the code are correct).
  2. **One report** – including explanations, mathematical derivations, and final results.
  3. Please do not ask an LLM to complete the entire homework for you.
  4. You may use LLM tools for guidance and assistance, but the work and implementation must be primarily your own.
  5. Also, do not use ready-made functions or external libraries that directly implement the required algorithms. You should implement the core methods yourself.
-